

Chapter 20: Context Compaction, Prompt Caching, and Token Budget Schedulers

Preceding Bridge: In [Chapter 16: Hierarchical Episodic Memory](#) and [Chapter 18: GPU Serving Mechanics & PagedAttention](#), you learned how LLM memory functions on the GPU. In this chapter, we explore how production agent architects handle massive long-context workloads efficiently: **Prompt Caching mechanics**, **The Lost-in-the-Middle Phenomenon**, and **Dynamic Context Compaction Algorithms (Observation Pruning & LLMingua)**.

1. Plain-English Jargon Demystifier

Technical Term	Plain English Translation	Real-World Metaphor
Prompt Caching	Reusing the pre-computed GPU KV-cache for identical prompt prefixes across subsequent API requests.	Keeping a thick reference textbook open on your desk so you don't have to re-read it from Page 1 every morning.
Cache Hit / Miss	Whether the initial tokens of your prompt exactly match an existing cached KV-cache block.	A fingerprint match at the passport border: match = instant green lane; mismatch = full manual search.
Lost-in-the-Middle	The empirical tendency of LLMs to recall information placed at the very start or end of a prompt, but forget facts in the center.	Remembering the intro and grand finale of a 3-hour movie, but completely forgetting what happened at minute 90.
Observation Pruning	Stripping verbose, raw tool outputs (e.g., a 5,000-line JSON payload) once the agent has extracted the core answer.	Erasing your rough chalkboard math calculations once you've written down the final equation.
LLMLingua	Using a small, fast model to calculate token information entropy and deleting 50% of low-information filler words.	Sending a concise telegram: <i>"Arriving noon send car"</i> instead of a flowery 3-paragraph letter.

2. Spoon-Fed Mental Model: The Open Textbook on the Desk

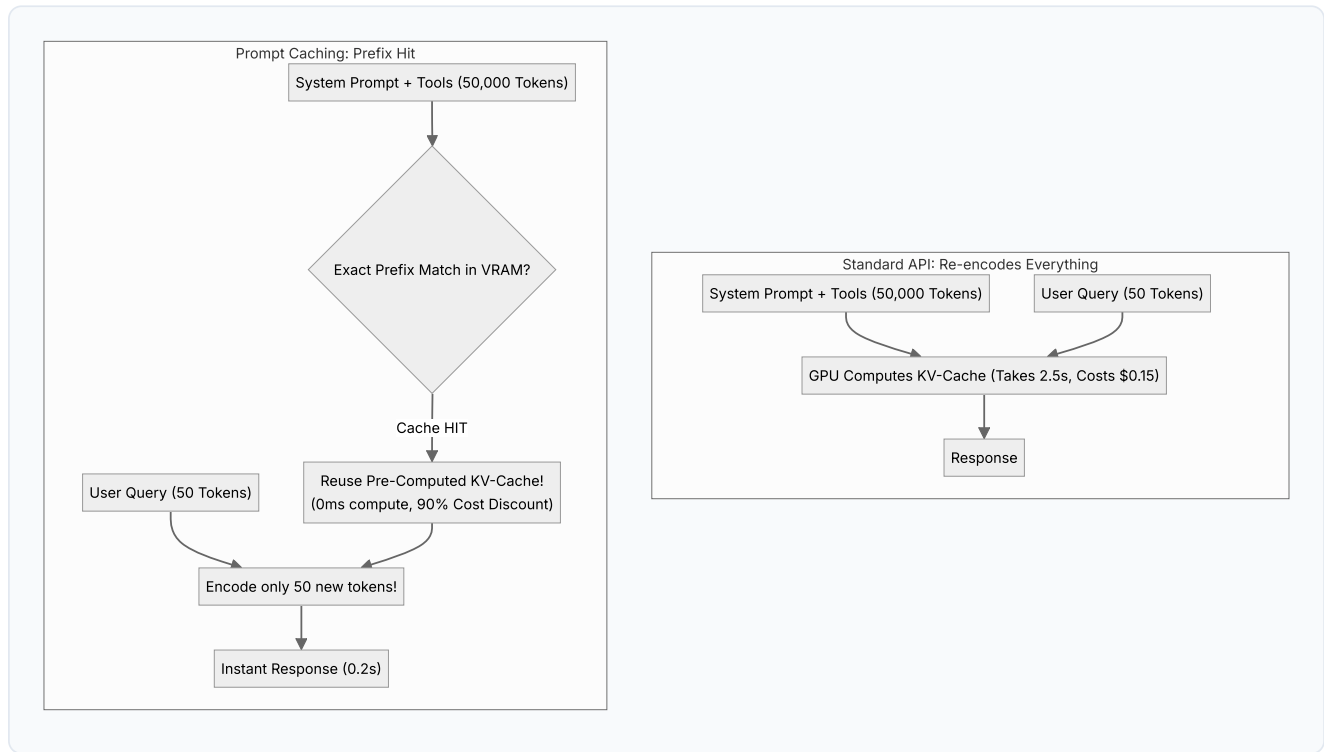
Imagine a research assistant tasked with answering questions based on a 400-page company legal handbook.

Without Prompt Caching (Naive Approach):

- **User 1 asks:** *"What is the parental leave policy?"* The assistant reads all 400 pages, finds the answer, and speaks.
- **User 2 asks:** *"What is the dental coverage limit?"* The assistant **re-reads all 400 pages from scratch**, wasting 10 minutes before speaking!
- **Cost:** Massive. **Latency:** High.

With Prompt Caching (Anthropic & OpenAI Mechanics):

- The assistant reads the 400-page handbook once, notes the exact page positions in memory, and **leaves the book open on the desk**.
- When User 2 arrives, the assistant skips the 400 pages entirely, reads only the 1-sentence question, and answers in **200 milliseconds!**
- Cost: **90% discount** on cached tokens.
- Latency: **80% reduction** in Time-to-First-Token (TTFT).



3. The 3 Architectural Rules of Prompt Caching

To guarantee a **100% Cache Hit Rate** in production agent frameworks:

1. **Prefix Invariance:** Caching operates strictly from the **very first character forward**. If you change a single word in the system prompt or reorder tool definitions, **every single token after that change suffers a Cache Miss!**
2. **Static-to-Dynamic Ordering:** Always structure prompts in order of least-frequently-changing to most-frequently-changing:

Static System Instructions → Tool Schemas → Few-Shot Examples → Dynamic User Conversation

3. **Minimum Token Threshold:** Providers enforce minimum token barriers (e.g., minimum 1,024 tokens for Anthropic Claude and OpenAI). Small prompts do not trigger caching.

4. Defeating "Lost-in-the-Middle": Context Window Compaction

Research reveals that in 128k+ token prompts, model attention forms a **U-shaped curve**: accuracy is highest near token 0 and token 128,000, and plummets in the middle 20% to 80% range!

Production Compaction Techniques:

1. **Observation Pruning:** When an agent runs `curl` or `fetch_sql` that returns 10,000 lines of data, the agent processes it into a 2-line conclusion. The full 10,000 lines are immediately replaced in history with: `[Raw payload pruned. Key takeaway: 42 records matched]`.
2. **Context Window Sliding Compactor:** When context exceeds 70% of maximum budget, older intermediate tool dialogue is compressed into a single deterministic summary block.

5. Complete Runnable Implementation: Prompt Cache Simulator & Context Compactor

Here is a 100% runnable, zero-dependency Python script demonstrating prompt prefix hash caching and dynamic observation compaction:

6. Chapter Milestone Check

Verify your understanding before continuing:

1. **Why does moving user-specific input to the top of a prompt destroy prompt caching?**
 - *Answer:* Prompt caching operates from the first character forward. Changing the start of the prompt generates a completely different prefix hash, causing an immediate cache miss on the entire prompt and forcing full re-computation.
2. **What causes the "Lost-in-the-Middle" phenomenon in 100k+ token prompts?**
 - *Answer:* Transformer self-attention mechanisms naturally place higher attention weights on initial positional tokens (primacy effect) and recent trailing tokens (recency effect), creating an attention trough in the middle of long contexts.
3. **What is observation pruning and when should an agent execute it?**
 - *Answer:* It is the practice of replacing large intermediate raw tool outputs (e.g. multi-megabyte JSON payloads or SQL table dumps) with concise summaries once the agent has finished parsing the necessary factual deductions, preserving context space for future turns.

Further Reading

- [Anthropic: prompt caching](#)
- [Lost in the middle paper](#)
- [LLMLingua paper](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. What does prompt caching require of your prompt layout?
- ▶ 2. What is context compaction?
- ▶ 3. What is 'lost in the middle'?
- ▶ 4. How do you budget tokens?